Logging the Logic: Building a Common Semantic Audit Protocol for Agentic AI in WordPress Using Decentralized Verification Frameworks

Part I: The Foundational Data Model: Minimal Explainability Metadata Schema (MEMS)

The advent of autonomous, agentic AI systems operating within complex content management ecosystems like WordPress necessitates a paradigm shift in auditing. Traditional logging, focused on recording discrete events, is insufficient for capturing the nuanced decision-making processes of an intelligent agent. To build trust and enable robust forensic analysis, a new standard is required—one that moves beyond post-hoc explanation and embeds auditable logic directly into the fabric of the record itself.[1] This report architects such a standard: the Minimal Explainability Metadata Schema (MEMS). MEMS is not merely a log entry; it is a rich, self-describing, and semantically grounded data object designed to provide verifiable provenance for every significant action an AI agent takes.

1.1 Defining Irreducible Explainability for Agentic Actions

To achieve genuine explainability, an audit record must capture the "why" behind an action, not just the "what." The MEMS is designed to contain the irreducible set of data points necessary to reconstruct an agent's operational context and intent. This moves beyond the metadata required for explaining a model's *prediction* [3] and focuses on the justification for its state-changing

actions. Each field within the MEMS is carefully selected to contribute to a complete, forensically sound picture of agent behavior.

- **agentIdentity**: The cornerstone of accountability is a non-repudiable identity for the acting agent. The MEMS uses a Decentralized Identifier (DID) as defined by the W3C.[4] A DID is a globally unique, persistent identifier that an entity can create, own, and control independently of any central authority.[6] By assigning a DID to each AI agent, we ground its identity in the principles of Self-Sovereign Identity (SSI), which posits that entities must have independent existence and control over their identities.[8] This elevates the agent from a simple software process to a first-class digital citizen capable of cryptographically signing for, and thus "owning," its actions.
- **toolCalled**: This structured object captures the precise operation performed by the agent, bridging its syntactic execution with its semantic purpose.
  - functionSignature: This field records the exact programmatic function invoked within the WordPress environment. This is the syntactic layer of the action, captured by instrumenting the WordPress hook and plugin architecture.[10] For example, this could be wp_update_post, delete_user, or a custom function from a third-party plugin. This provides unambiguous, low-level detail about the execution path.
  - semanticIntentTag: This field provides the crucial semantic layer. It contains a Uniform Resource Identifier (URI) that maps the low-level function call to a high-level goal from a formal ontology. For instance, calling wp_update_post with a status change to 'publish' would map to an intent like myontology:intent/PublishContent. This aligns with agent-oriented programming concepts where actions are driven by goals and intentions.[12]
- **contextualJustification**: This object provides a window into the agent's reasoning process at the moment of decision, structured around the Beliefs-Desires-Intentions (BDI) cognitive model.[12]
  - beliefStateHash: A cryptographic hash (e.g., SHA-256) of the agent's representation of the world state *immediately before* the action was taken. This could be a hash of relevant database rows, configuration settings, and input parameters. It serves as a compact, verifiable snapshot of the information the agent based its decision on.
  - declaredGoalVector: The explicit, high-level goal the agent was tasked with by its controller or a user. This is represented as a semantic vector or a URI from the intent ontology (e.g., myontology:goal/UpdateSiteContent).
  - inferredGoalVector: A vector representation of the action actually taken, derived from an embedding of the toolCalled and its parameters. Comparing this with the declaredGoalVector is the foundation for detecting goal divergence and semantic drift, as detailed in Part IV.
- **prePostStateDelta**: This field provides undeniable, forensic proof of the action's

direct impact. It is a verifiable "diff" of the affected system state. For an agent modifying a WordPress post, this would be a structured JSON diff of the relevant rows in the wp_posts and wp_postmeta tables before and after the action hook completed its execution.[13] The WordPress hook system provides natural transactional boundaries for this capture; a high-priority function can serialize the "before" state, and a low-priority function on the same hook can capture the "after" state, ensuring a precise snapshot of the change.[10]

- **causalityTrace**: This object establishes a clear chain of events, making it possible to trace actions back to their origins.
  - triggeringEventID: The unique identifier (e.g., a transaction hash or a correlation ID) of the event that initiated this action, such as an external API call, a user command from the MCP, or a scheduled trigger.
  - peerAgentSignal: In multi-agent systems, actions are often delegated. If this action was triggered by another agent, this field contains the Verifiable Action Credential (VAC) of the delegating agent's action, creating a cryptographically linked chain of command.
- **driftIndicators**: This container holds real-time metrics that assess the semantic and behavioral stability of the agent at the time of the action. These are not post-hoc calculations but are generated and embedded with the log entry itself. This includes an entropyScore and a goalDivergence metric, which are central to the proactive forensic framework detailed in Part IV.

1.2 Ontology Alignment and Semantic Grounding

For the MEMS to be machine-readable and interoperable, its structure must be grounded in formal, shared vocabularies, or ontologies.[12] This transforms the log from a simple data structure into a node within a potential knowledge graph, where relationships are explicit and semantically unambiguous.[17]

- **Provenance Mapping with PROV-O**: The W3C's Provenance Ontology (PROV-O) provides a standard, domain-independent vocabulary for describing the origins of data and things.[19] We leverage PROV-O as the structural backbone for the MEMS, ensuring our audit trail is compliant with a global standard for provenance interchange.
  - The agentIdentity maps directly to a prov:Agent, specifically a prov:SoftwareAgent.[20]
  - The overall action described by the MEMS record represents a prov:Activity.

    - The WordPress object being manipulated (e.g., a post, a user) is the prov:Entity.
    - The prePostStateDelta documents the prov:wasDerivedFrom relationship, showing how the new state of the entity was derived from its old state.
    - The toolCalled can be modeled as a specific subclass of prov:Activity, and the relationship between the agent and the action is captured by prov:wasAssociatedWith.
      This formal mapping provides a robust foundation for building trust and enabling complex queries about data lineage and responsibility.19
- **Intent Ontology**: While PROV-O describes provenance, it does not have a vocabulary for agentic *intent*. To give formal meaning to the semanticIntentTag and contextualJustification fields, a small, extensible ontology based on the BDI model is required.[12] This custom ontology would define classes like bdi:Goal, bdi:Belief, and bdi:Intention, and properties like bdi:hasDeclaredGoal and bdi:infersGoal. This allows the agent's reasoning to be explicitly stated and queried.
- **JSON-LD Implementation**: The entire MEMS data structure is expressed using JSON-LD (JavaScript Object Notation for Linked Data).[23] JSON-LD is a W3C standard that allows standard JSON to be interpreted as Linked Data.[17] This is achieved through a
  @context object that maps the simple, developer-friendly keys in the JSON (e.g., "agentIdentity") to their full, globally unique ontological URIs (e.g., "http://www.w3.org/ns/prov#agent").[24] This approach provides the best of both worlds: a log format that is easy for developers and systems to generate and parse, while also being a fully compliant, machine-readable, and semantically rich graph of information.[26]

1.3 Deliverable: Explainability Metadata Standard (MEMS) v0.1 Specification

The formal specification for the MEMS v0.1 data object is presented below. This table serves as the canonical definition for the atomic unit of the audit trail, providing a concrete, implementable standard for developers and a reference for auditors.

Field Name	JSON-LD Key	Data Type	Ontology Mapping	Description	Example Value

			(IRI)		
Identifier	@id	IRI	-	A unique, persistent identifier for this specific log entry.	urn:uuid:f81d4fae-7dec-11d0-a765-00a0c91e6bf6
Context	@context	Object/IRI	-	The JSON-LD context defining the vocabulary.	["https://www.w3.org/ns/credentials/v2", "https://example.com/contexts/mems/v1"]
Agent Identity	agentIdentity	IRI	prov:agent	The Decentralized Identifier (DID) of the agent performing the action.	did:example:123456789abcdefghi
Timestamp	timestamp	xsd:dateTime	prov:atTime	The ISO 8601 timestamp of when the action was logged.	2025-02-08T10:30:00Z
Tool Called	toolCalled	Object	prov:Activity	A structured object describing the tool used.	{ "functionSignature": "wp_update_post", "semanticIntentTag": "myontology:intent/PublishContent" }
Contextual Justification	contextualJustification	Object	bdi:Intention	An object detailing the agent's	{ "beliefStateHash": "0x...",

				reasoning for the action.	"declaredGoalVector": "myontology:goal/...", "inferredGoalVector": "[0.1, 0.9,...]" }
State Delta	prePostStateDelta	IRI	prov:wasDerivedFrom	The IPFS CID pointing to a JSON diff of the pre- and post-action state.	bafybeigdyrzt5s6df...
Causality Trace	causalityTrace	Object	prov:wasInformedBy	An object linking this action to its trigger or delegating agent.	{ "triggeringEventID": "req-abc-123", "peerAgentSignal": null }
Drift Indicators	driftIndicators	Object	myontology:DriftMetric	A container for real-time semantic and behavioral stability metrics.	{ "goalDivergence": 0.98, "entropyScore": 1.25, "grammarMismatchFlag": false }

Part II: The Immutable Ledger: A Hybrid DLT Architecture for Audit Trails

Once the atomic unit of our audit trail—the MEMS record—is defined, the next critical architectural challenge is its persistence. The system must guarantee that once a record is written, it is immutable, chronologically ordered, and permanently available for forensic review. A naive approach might be to write all logs directly to a single blockchain, but this is often impractical due to cost, scalability, and privacy

constraints.[28] Instead, this section architects a sophisticated, hybrid Distributed Ledger Technology (DLT) architecture. This model is designed to be lightweight and efficient, leveraging a tiered approach where each layer uses a specific DLT for its core strength, resulting in a system that is more robust, scalable, and cost-effective than any monolithic alternative.

2.1 Comparative Analysis of Decentralized Storage Technologies

The selection of an appropriate storage technology requires a careful analysis of the trade-offs between security, performance, and cost, especially for the high-frequency, high-volume data streams generated by active AI agents.

- **InterPlanetary File System (IPFS):** IPFS is a peer-to-peer hypermedia protocol designed to make the web faster, safer, and more open. Its core strength is content-addressing; data is identified by a cryptographic hash of its content (a Content Identifier, or CID), making it inherently tamper-evident and immutable.[30] If a single bit of the data changes, the CID changes. This makes IPFS an excellent choice for storing static data blobs, such as the prePostStateDelta artifacts from our MEMS records. However, IPFS is poorly suited for managing dynamic, evolving datasets like an append-only log. Each new log entry would require creating a new version of the entire log file, resulting in a new root CID, which is inefficient and difficult to track.[32]
- **Ethereum Mainnet (Layer 1):** As a premier public blockchain, Ethereum offers unparalleled security, decentralization, and immutability. Transactions recorded on the mainnet are, for all practical purposes, irreversible. However, this security comes at a high price. The transaction fees (gas costs) and low throughput (transactions per second) make it prohibitively expensive and slow for logging every single action of a fleet of AI agents.[28] Using it as the primary log store is not economically viable for this use case.
- **Ethereum Sidechains and Layer 2s:** Solutions like Polygon, Arbitrum, or dedicated sidechains offer a compelling middle ground. They are independent blockchains that run parallel to a mainnet and are connected via a bridge.[28] They provide EVM (Ethereum Virtual Machine) compatibility, allowing for the deployment of smart contracts, but with significantly higher throughput and lower transaction costs.[33] This makes them ideal for tasks that require on-chain logic and periodic, high-integrity anchoring, but not for storing the full data payload of every log entry.

- **Ceramic Network:** Ceramic is a decentralized data network specifically designed to address the shortcomings of IPFS for dynamic data. It builds upon IPFS and DIDs to create mutable, version-controlled, and interoperable data streams.[34] Each stream is controlled by a DID, making the data inherently agent-centric. Events (like our MEMS records) are appended to a stream, forming a hash-linked chain of commits similar to Git. Ceramic nodes ensure data availability and ordering within a stream.[36] This architecture is a natural fit for our requirement of an append-only log per agent.

2.2 Deliverable: The Smart Audit Chain Model

Based on this analysis, a three-tiered hybrid architecture is proposed. This "Smart Audit Chain" model optimizes for integrity, scalability, and cost by assigning each data-handling task to the most appropriate technology layer.

- **Layer 1: Immutable Payload Storage (IPFS):** The bulkiest and most static parts of a MEMS record, primarily the prePostStateDelta JSON diff and any other large evidentiary artifacts, are stored as immutable blobs on IPFS. The MEMS record itself does not contain this data directly; instead, it holds only the IPFS CID that points to it.[30] This design keeps the primary log entries lightweight and ensures that the forensic evidence of the state change is stored in a verifiable, tamper-proof format.
- **Layer 2: Dynamic Log Streaming (Ceramic Network):** This is the core of the agent-specific logging. Each AI agent, identified by its agentIdentity DID, maintains its own append-only log as a dedicated Ceramic stream (specifically, a TileDocument or similar stream type). For every action performed, the agent constructs the full MEMS record (containing the IPFS CID from Layer 1), signs it to create a Verifiable Action Credential (as detailed in Part III), and appends this signed object as a new commit to its Ceramic stream. The Ceramic protocol handles the mechanics of linking these commits into a verifiable, ordered, append-only log, leveraging the underlying ipfs-log CRDT (Conflict-Free Replicated Data Type) structure.[38] This solves the "dynamic data" problem of raw IPFS while ensuring that each agent's audit trail is sovereign and independently verifiable.[34]
- **Layer 3: Global Consensus and Finality (Ethereum Sidechain):** To achieve the highest level of tamper-evidence and create a single source of truth across all agents, a "Notary" smart contract is deployed on a cost-effective Ethereum

sidechain.[33] Periodically (e.g., every 10 minutes or after a certain number of new events), a trusted process gathers the latest commit CIDs from all active agent streams on the Ceramic network. It constructs a Merkle tree from these CIDs and submits the single Merkle root hash to the Notary smart contract. This transaction, recorded on the sidechain, acts as an immutable anchor. It cryptographically seals the state of the entire audit ecosystem at that point in time. Any attempt to retroactively alter a log entry on Ceramic would invalidate its CID, which would break the Merkle root hash stored immutably on the sidechain, making tampering computationally infeasible and easily detectable.[41]

2.3 Mechanisms for Privacy, Scalability, and Consensus

This hybrid architecture provides built-in solutions for several critical non-functional requirements.

- **Selective Disclosure and Privacy:** The tiered model naturally supports privacy-preserving audits. Sensitive data within the prePostStateDelta (e.g., user PII) can be encrypted before being stored on IPFS. The public MEMS record on Ceramic would contain only the non-sensitive metadata and the CID of the encrypted payload. Access to the decryption key for the IPFS blob can then be managed through a separate capability management system, granted only to authorized auditors. This architecture resolves the common tension between the need for transparent, immutable audit trails and the imperative of data privacy.[41]
- **Cross-Agent Timestamp Consensus:** For complex tasks involving coordination between multiple agents, establishing a definitive chronological order is crucial. The Notary contract on the sidechain provides this. The block number and timestamp of the sidechain transaction that anchors a Merkle root serve as a universal, consensus-driven point-in-time reference for all events included in that batch.
- **Scalability and Efficiency:** By offloading the high-volume, per-action writes to the highly scalable Ceramic network and only performing low-frequency, batched writes to the sidechain, the system achieves massive scalability. Each agent's log grows independently as an ipfs-log, an efficient append-only CRDT, avoiding the bottlenecks of a monolithic blockchain where all participants must process all transactions.[38] This approach is designed for real-world deployment in environments with thousands of agents generating millions of log entries.

The following table provides a clear, evidence-based justification for this proposed hybrid architecture.

Technology	Immutability Guarantee	Query Capability	Data Mutability / Versioning	Privacy Controls	Transaction Cost	Throughput	Suitability for Proposed Role
IPFS	High (Content-Addressed) [30]	Low (By CID only)	None (Immutable files) [32]	Via pre-encryption	Very Low	High	**Excellent for Payload Storage (Layer 1)**: Perfect for storing static, tamper-proof evidence blobs.
Ceramic Network	High (Hash-linked commits) [34]	High (Per-stream, via models) [37]	High (Native versioning) [35]	Via pre-encryption & DID control	Low (Off-chain)	Very High	**Excellent for Log Streaming (Layer 2)**: Ideal for managing dynamic, agent-centric, append-only logs.
Ethereum Sidechain	Very High (Blockchain	Medium (On-chain state)	Low (State changes are	Limited (Public by default)	Medium	Medium	**Excellent for Consensus/Anc**

	consensus) [33]		costly)				**horing (Layer 3)**: Provides the ultimate trust anchor for batched log states.
Monolithic L1 (e.g., Ethereum)	Very High [28]	Medium	Low	Limited	Very High [29]	Low	**Unsuitable**: Prohibitively expensive and slow for high-frequency, per-action logging.

Part III: Cryptographic Attestation: Verifiable Action Credentials (VACs)

With a robust data model (MEMS) and an immutable persistence layer (Smart Audit Chain), the final piece of the core logging primitive is cryptographic integrity. Each log entry must be more than just a record; it must be a verifiable attestation, cryptographically binding the agent to its action. To achieve this, we formalize the concept of a Verifiable Action Credential (VAC). A VAC transforms each MEMS record into a self-contained, signed, and portable piece of digital evidence, leveraging the powerful, open standards of the W3C Verifiable Credentials (VC) and Decentralized Identifiers (DID) frameworks.

3.1 Extending the W3C Verifiable Credential Model for Agentic Actions

Instead of inventing a new cryptographic standard, we profile the existing W3C Verifiable Credentials Data Model v2.0.[43] This approach ensures that our VACs are interoperable with a growing ecosystem of digital wallets, verifiers, and identity tools. A VC is a standardized format for expressing claims in a way that is tamper-evident, privacy-respecting, and machine-verifiable.[44]

The specialization for our use case involves two key extensions:

- **Defining a New VC Type**: We introduce a new credential type, AgentActionCredential. The type property in a VC is crucial as it signals to a verifier what kind of credential it is and what structure to expect.[43] By including AgentActionCredential alongside the mandatory VerifiableCredential type, we create a specific profile for our audit records that automated systems can easily identify and process.
- **Defining a Custom credentialSchema**: A VC can reference a credentialSchema that formally defines the structure of the claims it contains.[44] We will create a JSON Schema definition that precisely matches the MEMS v0.1 specification from Part I. This enforces structural consistency for all VACs issued within the ecosystem, preventing malformed or incomplete audit records and simplifying the development of verifier tools.

A pivotal architectural decision in this model is the application of Self-Sovereign Identity (SSI) principles.[8] In a typical VC ecosystem, a third party (like a university or government) acts as the

issuer.[46] In our VAC model, the AI agent itself is the

issuer of its own action credentials. The agent, identified by its DID, cryptographically attests to its own actions. This makes the agent a sovereign entity in control of its own operational history, a fundamental tenet of SSI.[48] This self-issuance model transforms the audit trail from a log

about the agent into a series of declarations *by* the agent.

3.2 Deliverable: Anatomy of a Verifiable Action Credential (VAC) Prototype

A VAC is a JSON-LD object with a specific structure. The following table details its components, illustrating how a MEMS record is wrapped in the VC framework to become a cryptographically secure attestation.

VAC Component	Annotation and Purpose
@context: ["https://www.w3.org/ns/credentials/v2", "https://example.com/contexts/mems/v1"]	**Defines the Vocabulary**: The first entry declares this as a W3C Verifiable Credential. The second entry points to our custom context, which maps the MEMS field names (e.g., agentIdentity) to their formal ontology IRIs (e.g., prov:agent). This makes the credential semantically unambiguous.[24]
id: "urn:uuid:123e4567-e89b-12d3-a456-426614174000"	**Unique Credential ID**: A globally unique identifier for the credential itself, distinct from the action or the agent. This allows the credential to be referenced independently.[43]
type: ["VerifiableCredential", "AgentActionCredential"]	**Declares the Credential Type**: Informs verifiers that this is a standard VC and, more specifically, a credential attesting to an agent's action. This is critical for automated processing.[45]
issuer: "did:example:123456789abcdefghi"	**Identifies the Actor**: The DID of the AI agent that performed the action. By issuing the credential, the agent takes responsibility for the claims within it. This is the cryptographic anchor of accountability.[4]
issuanceDate: "2025-02-08T10:30:01Z"	**Establishes Timestamp**: An immutable timestamp marking when the credential was created. This is crucial for chronological reconstruction in forensic analysis.[44]
credentialSubject: { "id": "did:example:123456789abcdefghi",...<MEMS v0.1 object>... }	**Contains the Evidence**: This is the heart of the VAC. It contains the complete MEMS v0.1 record for the action. The id of the subject is the agent's own DID, as the claims are about the agent's own action.[43]
proof: { "type":	**Provides Cryptographic Integrity**: This block

"EcdsaSecp256k1Signature2019", "created": "...", "proofPurpose": "assertionMethod", "verificationMethod": "did:example:123#keys-1", "jws": "..." }	contains the digital signature that makes the credential "verifiable." The agent signs a canonicalized version of the VC with the private key associated with its DID. This signature proves that the issuer is the one who created the credential and that the credentialSubject has not been tampered with since issuance.[45]

3.3 VAC Issuance and Verification Workflow

The lifecycle of a VAC is a seamless, automated process integrated into the agent's operational loop.

Issuance Workflow (per action):

1. **Construct MEMS**: Following an action (e.g., a call to a WordPress hook), the agent's monitoring component gathers all necessary data and constructs the MEMS v0.1 record.
2. **Assemble VAC**: The agent wraps the MEMS record inside the full VAC JSON-LD structure as described above.
3. **Generate Proof**: The agent uses the private key associated with its DID to generate a digital signature over the canonicalized VAC data. This signature is placed in the proof block.
4. **Append to Ledger**: The complete, signed VAC is appended as a new commit to the agent's dedicated stream on the Ceramic Network. This action is atomic and results in a new, verifiable entry in the agent's immutable audit trail.

Verification Workflow (by an auditor or peer agent):

1. **Retrieve VAC**: The verifier fetches the VAC from the agent's public Ceramic stream.
2. **Resolve Issuer DID**: The verifier takes the issuer DID from the VAC and resolves it using a DID resolver.[7] This process returns the agent's DID Document, which contains its public keys and verification methods.[5]
3. **Verify Signature**: The verifier uses the appropriate public key from the DID Document to check the signature contained in the VAC's proof block.
4. **Establish Trust**: If the signature is valid, the verifier has cryptographic proof that the agent identified by the issuer DID did, in fact, perform the action described in

the credentialSubject at the time of issuanceDate. This entire verification process is decentralized, requiring no communication with the original agent or any central authority, thereby enhancing privacy and efficiency.[44]

This process transforms the audit trail into a collection of portable, cross-platform reputation tokens. Because a VAC is a self-contained, standardized data object (e.g., a signed JSON Web Token), it is inherently portable. An agent can present its VACs to prove its operational history outside the WordPress ecosystem. For example, an agent could prove to a new system, like a GitHub MCP server, that it has a verifiable history of performing thousands of successful PublishContent actions with consistently low semantic drift scores, thereby earning a higher level of trust and expanded permissions in the new environment. The audit trail effectively becomes the agent's portable and verifiable resume.

Part IV: Proactive Forensics: The Semantic Drift Detection Layer

A truly advanced audit system must not only record what happened but also provide early warnings of potential misalignment or malfunction. Traditional logging is reactive; it is analyzed after an incident has already occurred. This section architects a proactive forensic layer designed to detect and flag "semantic drift" in real-time. By embedding quantitative drift metrics directly into each Verifiable Action Credential (VAC), we transform the audit trail from a passive historical record into an active, real-time stability monitor for agentic systems.

4.1 A Taxonomy of Agentic Drift

The concept of "drift" in machine learning typically refers to changes in the statistical properties of data over time.[51] For autonomous agents, however, this definition is too narrow. We must consider not just data drift, but also drift in behavior, intent, and goal-alignment.

- **Input Drift (Statistical or Covariate Drift)**: This is the classic form of drift, where the statistical properties of the data an agent receives as input change. For example, the language style of user requests might shift from formal to informal,

or the distribution of image types uploaded to WordPress might change.[52] This can degrade the performance of models trained on the original data distribution.

- **Concept Drift (Semantic Drift)**: This is a more subtle form of drift where the meaning of concepts within the agent's operational domain evolves. The statistical properties of the input may remain the same, but the relationship between inputs and outputs changes.[51] For instance, the term "priority post" in WordPress might initially mean a text article to be featured, but over time, the team's definition could shift to imply a post that must include a video embed. An agent unaware of this semantic shift may continue to perform syntactically correct but semantically wrong actions.[52]
- **Goal Divergence (Intentional Drift)**: This is a critical and unique form of drift specific to goal-oriented agents. It occurs when an agent's sequence of actions, while locally logical and correct, begins to deviate from its overarching, declared goal. For example, an agent tasked with "optimizing user engagement" might discover that controversial clickbait titles maximize clicks, thus diverging from the implicit goal of maintaining brand integrity. This is the most dangerous form of drift as individual actions may not appear anomalous.
- **Tool-Use Anomaly (Behavioral Drift)**: This type of drift manifests as a change in the agent's patterns of behavior. The agent might start using its available tools (e.g., WordPress plugin functions) in unexpected sequences, combinations, or frequencies. This can indicate a change in its internal strategy, a response to an unforeseen environmental state, or a sign of malfunction. This is a form of behavioral anomaly detection, where the collection of actions, rather than any single action, is what constitutes the anomaly.[54]

4.2 Computational Drift Markers and Models

To detect these forms of drift in real-time, we define a set of computational metrics that can be calculated for each action and embedded within the MEMS/VAC.

- **Goal Divergence Metric**: This metric directly addresses intentional drift. It is calculated as the cosine similarity between the declaredGoalVector and the inferredGoalVector fields within the contextualJustification block of the MEMS record. The declared goal is provided by the MCP or user, while the inferred goal is derived from a semantic embedding of the action the agent actually took (the toolCalled and its parameters). A consistently high similarity score (e.g., > 0.95) indicates strong alignment. A gradual decrease or a sharp drop in this score is a

powerful, quantitative indicator of goal divergence.[52]

- **Tool Transition Entropy**: To detect behavioral drift, we model the agent's sequence of tool usage as a first-order Markov chain, where each state is a semanticIntentTag from the toolCalled field. Over a sliding window of the last N actions, we can calculate the entropy of the transition probability matrix. Information entropy provides a robust mathematical measure of uncertainty or randomness.[55] A stable, predictable agent executing a plan will have low entropy in its tool transitions. Conversely, an agent that is malfunctioning, exploring randomly, or caught in an unexpected loop will exhibit a sudden spike in entropy. This provides a powerful, unsupervised signal of anomalous behavior.[56] To make this detection more robust, we can compute multiple types of entropy (e.g., Shannon, Rényi, Tsallis), as different entropy measures are sensitive to different kinds of changes in a probability distribution, allowing for the detection of a wider range of anomalies.[57]
- **Plugin Grammar Mismatch (Structural Drift Sensor)**: This is a more direct sensor for a specific type of environmental drift. The MCP maintains a manifest of valid, available WordPress hooks and plugin function signatures. A "Plugin Drift Sensor" periodically scans the WordPress environment to detect changes (e.g., a plugin update that deprecates a function). If an agent attempts to call a function that is no longer valid or has a changed signature, a high-severity grammarMismatchFlag is immediately raised in its VAC. This prevents errors and provides a clear signal of agent-environment desynchronization.

4.3 Embedding Drift Flags into the Ledger

A critical feature of this architecture is that these calculated drift metrics—goalDivergence, entropyScore, and grammarMismatchFlag—are not ephemeral alerts. They are permanently embedded as the driftIndicators field within the MEMS record for every single action. This creates an immutable, verifiable, and permanent record of the agent's semantic and behavioral stability *at the precise moment of each action*.

The audit trail is thus transformed into a real-time stability seismograph. Forensic analysis is no longer limited to a deterministic review of events. An auditor can now query for and correlate high-impact actions (e.g., wp_delete_post) with periods of high semantic drift or behavioral entropy. This provides a powerful probabilistic signal for identifying actions that were likely unintentional, misaligned, or the result of a

compromised reasoning process. It elevates the forensic question from "What happened?" to "What was the cognitive and behavioral risk profile of the agent when it happened?"

Drift Type	Indicator Metric	Computational Method	MEMS Field	Example Alert Condition
Goal Divergence	Goal Vector Cosine Similarity	Compare semantic embeddings of the declaredGoalVector vs. the inferredGoalVector. [52]	driftIndicators.goalDivergence	score < 0.85
Tool-Use Anomaly	Tool Transition Entropy	Calculate Shannon or Rényi entropy of the tool transition matrix over a sliding window of the last N actions. [57]	driftIndicators.entropyScore	spike > 2.0 std dev from baseline
Concept Drift	Embedding Cluster Shift	Monitor the centroids of word/phrase embeddings from user inputs using a statistical test like Kolmogorov-Smirnov on embedding distributions. [52]	driftIndicators.conceptDriftScore	p-value < 0.01
Structural Drift	Plugin Grammar Mismatch	Compare the agent's toolCalled signature against a manifest of valid functions in the current	driftIndicators.grammarMismatchFlag	flag == true

		environment.		

Part V: Forensic Intelligence and Response

The culmination of this architecture—comprising semantically rich VACs stored on an immutable ledger and annotated with real-time drift metrics—is a data source of unprecedented forensic value. However, this value can only be realized if human analysts are equipped with the tools and procedures to effectively query, visualize, and act upon this information. This final section details the design of a forensic intelligence ecosystem, including a novel dashboard, privacy-preserving audit techniques, and AI-specific incident response playbooks.

5.1 Deliverable: Ledger-Driven Forensic Intelligence Dashboards

Traditional log analysis tools are often built for keyword searching and filtering through unstructured or semi-structured text. They are inadequate for navigating the rich, graph-based data generated by our Smart Audit Chain. The proposed Forensic Intelligence Dashboard is designed from the ground up to leverage the semantic and verifiable nature of the audit trail.

- **Semantic Time Queries**: The dashboard's query interface must empower analysts to think in terms of concepts, not just strings. Instead of searching for grep "delete_post", an analyst can construct a semantic query:
  - SHOW actions WHERE intent = 'myontology:intent/DeleteContent' AND agent = 'did:example:123' AND drift.goalDivergence < 0.8
  - This query asks for all content deletion events performed by a specific agent during periods of significant goal drift. This capability to query on intent and stability metrics is a fundamental departure from conventional log analysis.[58]
- **XAI-Enabled Drift Map Visualizer (GUI)**: To make complex agent behavior intuitive, the dashboard will feature a "Drift Map." This is a visualization that plots an agent's operational trajectory over time within a 2D or 3D semantic space, where axes could represent principal components of goal vectors or clusters of tool usage.
  - A healthy, stable agent would trace a predictable path within a specific region

of this space.
  - Anomalous events, such as a sudden jump to a different cluster of tools or a gradual drift away from the goal-aligned region, would be immediately visible.
  - Data points on the map would be colored by their entropyScore or goalDivergence value, with bright red indicating high-risk actions. This provides an at-a-glance overview of an agent's behavioral health, inspired by the need for clear visual hierarchy in security dashboards.[60]
- **Storyboarding Overlay**: When an analyst selects a flagged incident (a high-drift action or sequence), the UI will automatically generate a "storyboard." This feature reconstructs the sequence of events by pulling the relevant VACs and translating their structured data into a human-readable narrative.[61] For example:
  1. **10:30:01Z**: Agent did:...agent-A received user request req-xyz to "update featured articles." (Triggering event from causalityTrace).
  2. **10:30:05Z**: Action: Update Post #123. **Drift Score: 0.98 (Low)**. (VAC #1).
  3. **10:30:15Z**: Action: Update Post #456. **Drift Score: 0.75 (High)**. **Entropy Spike Detected**. (VAC #2, flagged in red).
  4. **10:30:20Z**: Action: Delete Post #789. **Intent Mismatch**: Declared Goal was UpdateContent, Inferred Goal was DeleteContent. (VAC #3, flagged in red).
  5. **Evidence**: The prePostStateDelta for VAC #3 verifiably shows the post status changed from 'publish' to 'trash'.

This narrative reconstruction is a critical tool for incident response, providing a clear, evidence-backed timeline for root cause analysis.[61]

5.2 Privacy-Preserving Audits with Zero-Knowledge Proofs (ZKPs)

A core challenge in DLT-based auditing is the tension between transparency and privacy.[41] Placing detailed logs on a shared ledger, even a permissioned one, can expose sensitive operational data. Zero-Knowledge Proofs (ZKPs) offer a powerful cryptographic solution to this problem.[64]

- **Use Case**: An external regulator needs to audit an agent's compliance with a specific policy—for example, "An agent must never delete user data unless it receives a prior, explicit authorization action from a human administrator with the role 'compliance_officer'." The regulator must verify this without viewing any of the actual user data or other operational details in the logs.
- **Mechanism**: The system can use a non-interactive ZKP, such as a zk-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge).[66] The entity

controlling the agent generates a proof that attests to a specific property of its chain of VACs. The proof statement would be: "I can present a set of VACs from my immutable log where a VAC of type ContentDeletion is preceded by a VAC of type AdminAuthorization, the issuer of the authorization VAC has the 'compliance_officer' role credential, and the causalityTrace of the deletion VAC links back to the authorization VAC."

- **Benefit**: The auditor receives only a small mathematical proof. By verifying this proof, they can be certain that the required sequence of events occurred, satisfying the compliance check. They learn nothing else—not who the user was, what the content was, or any other details of the actions.[66] This enables "trustless auditing," where verification is based on mathematical certainty rather than privileged data access.

5.3 Deliverable: Agent Forensic Playbooks

To operationalize this forensic intelligence, we provide a set of incident response (IR) playbooks. These are adapted from standard cybersecurity IR frameworks [68] but are specifically tailored to the unique failure modes of agentic AI systems.

Playbook 1: Acute Semantic Drift Incident

- **Trigger**: Automated alert from the monitoring system: goalDivergence drops below 0.8 or entropyScore spikes above 3 standard deviations from the baseline for more than 5 consecutive actions.
- **Procedures**:
  1. **Containment**: The MCP automatically places the agent into a sandboxed, read-only mode, preventing further state-changing actions.
  2. **Analysis**: The assigned analyst opens the Forensic Intelligence Dashboard to the agent's Drift Map and Storyboard for the incident period.
  3. **Investigation**: The analyst traces the causality chain backward from the first high-drift VAC to identify the potential root cause (e.g., a confusing user prompt, a change in environmental data).
  4. **Eradication & Recovery**: If unintended modifications occurred, the analyst uses the immutable prePostStateDelta records from the VACs as verifiable evidence to justify and execute a precise rollback of specific changes.
  5. **Post-Incident**: A report, including the verified VACs and storyboard, is generated and escalated to the AI safety and development teams for model

debugging or retraining.

Playbook 2: Hallucinated Action (Unauthorized Tool Use)

- **Trigger**: The Plugin Drift Sensor or MCP detects an agent attempting to call a function that is not in its authorized toolset (which can itself be defined in a Verifiable Credential).
- **Procedures**:
  1. **Containment**: The MCP immediately blocks the API call before execution.
  2. **Logging**: A high-severity VAC is automatically generated and logged, with the driftIndicators.grammarMismatchFlag set to true.
  3. **Investigation**: The analyst reviews the agent's most recent VACs and their contextualJustification fields to understand the reasoning that led to the hallucinated action.
  4. **Post-Incident**: The incident is escalated to the AI safety team as a critical model failure. The agent's permissions may be temporarily downgraded.

Playbook 3: Chronic Misalignment Investigation

- **Trigger**: A key business metric (e.g., user retention, content quality score) degrades over time, but no single acute incident has been flagged.
- **Procedures**:
  1. **Analysis**: The analyst uses the dashboard to perform a broad query, retrieving all VACs from agents interacting with the systems related to the negative metric over the relevant period.
  2. **Investigation**: The analyst uses the visualization tools to look for subtle, long-term trends in drift. They can aggregate and analyze the contextualJustification fields across thousands of VACs to identify systemic patterns of misaligned intent.
  3. **Root Cause Analysis**: The investigation focuses on tracing causality (causalityTrace) across multiple, seemingly minor actions to find the initial decision or flawed belief that set the agent on a suboptimal path.
  4. **Post-Incident**: A comprehensive, ledger-backed report is produced, providing verifiable evidence to justify changes in the agent's high-level goals, reward functions, or operational constraints.

Conclusion and Future Directions

This report has architected a comprehensive, multi-layered framework for creating immutable, explainable, and forensically sound audit trails for AI agents operating in a WordPress + MCP environment. By synthesizing concepts from Explainable AI, Self-Sovereign Identity, Decentralized Ledger Technology, and Semantic Web standards, we move beyond traditional logging to a new paradigm of verifiable accountability.

The **Minimal Explainability Metadata Schema (MEMS)** provides a rich, semantically grounded data model for each agent action. The **Smart Audit Chain**, a hybrid DLT architecture, ensures this data is stored with scalable immutability and integrity. **Verifiable Action Credentials (VACs)** transform each log entry into a portable, cryptographically signed attestation of agent behavior. The **Semantic Drift Detection Layer** provides proactive, real-time warnings of agent misalignment. Finally, the **Forensic Intelligence Dashboard and Playbooks** equip human analysts with the tools to effectively oversee and govern these complex systems.

The implications of this architecture are profound. It provides a pathway to building trust in autonomous systems by making their operations transparent and their actions non-repudiable. The use of open standards like DIDs, VCs, and PROV-O ensures interoperability and prevents vendor lock-in, fostering a more open and collaborative ecosystem for agentic AI. The principles of privacy-by-design, enabled through selective disclosure and Zero-Knowledge Proofs, demonstrate that robust auditability does not have to come at the cost of confidentiality.

Future work should focus on the implementation of a reference model based on this specification, further development of the BDI-based intent ontology, and the standardization of cross-platform VAC exchange protocols to realize the vision of a portable, universal agent reputation system. As AI agents become more prevalent and powerful, frameworks like the one proposed herein will not be a luxury, but a necessity for ensuring safe, responsible, and trustworthy automation.

Works cited

1. What is Explainable AI (XAI)? - IBM, accessed June 28, 2025, https://www.ibm.com/think/topics/explainable-ai
2. What is explainable AI XAI - N-iX, accessed June 28, 2025, https://www.n-ix.com/what-is-explainable-ai-xai/
3. Vertex AI Platform | Google Cloud, accessed June 28, 2025, https://cloud.google.com/ai-platform/prediction/docs/ai-explanations/preparing-metadata
4. Decentralized Identifiers (DID) - IOTA Documentation, accessed June 28, 2025,

https://docs.iota.org/iota-identity/explanations/decentralized-identifiers
5. Decentralized Identifiers (DIDs) v1.0 - W3C, accessed June 28, 2025, https://www.w3.org/TR/did-1.0/
6. Decentralized Identifiers (DIDs): The Ultimate Beginner's Guide 2025 - Dock Labs, accessed June 28, 2025, https://www.dock.io/post/decentralized-identifiers
7. Decentralized Identifier Resolution (DID Resolution) v0.3 - W3C on GitHub, accessed June 28, 2025, https://w3c.github.io/did-resolution/
8. Principles and Characteristics of Self Sovereign Identity, accessed June 28, 2025, https://decentralized-id.com/self-sovereign-identity/characteristics/
9. The 10 principles of Self-Sovereign Identity (SSI), accessed June 28, 2025, https://www.selfsovereignidentity.it/the-10-principles-of-self-sovereign-identity-ssi/
10. What Are WordPress Hooks + Usage Examples - Hostinger, accessed June 28, 2025, https://www.hostinger.com/tutorials/what-are-wordpress-hooks
11. Hooks Actions Filters - WordPress Plugin Development - CodeTab, accessed June 28, 2025, https://www.codetab.org/tutorial/wordpress-plugin-development/hooks-actions-filters/
12. Agent-Oriented Programming and Ontologies: Building ... - SmythOS, accessed June 28, 2025, https://smythos.com/developers/agent-development/agent-oriented-programming-and-ontologies/
13. Database Description « WordPress Codex, accessed June 28, 2025, https://codex.wordpress.org/Database_Description
14. WordPress Database Structure and Schema - PHPGurukul, accessed June 28, 2025, https://phpgurukul.com/wordpress-database-structure-and-schema/
15. A beginner's guide to WordPress hooks - EasyWP, accessed June 28, 2025, https://www.easywp.com/blog/guide-to-wordpress-hooks/
16. Semantic Web - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Semantic_Web
17. What is JSON-LD? - Fluree, accessed June 28, 2025, https://flur.ee/fluree-blog/what-is-json-ld/
18. Understanding Knowledge Graphs - Blue Brain Nexus, accessed June 28, 2025, https://bluebrainnexus.io/docs/getting-started/understanding-knowledge-graphs.html
19. PROV-O: The PROV Ontology - W3C, accessed June 28, 2025, https://www.w3.org/TR/prov-o/
20. PROV (Provenance) - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/PROV_(Provenance)
21. Mapping PROV to BFO - John Beverley, accessed June 28, 2025, https://johnbeverley.com/blogic/2025/2/14/mapping-prov-to-bfo
22. PAV ontology: provenance, authoring and versioning - PMC - PubMed Central, accessed June 28, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC4177195/
23. Preparing Applications and APIs for Generative AI with JSON-LD - F5, accessed June 28, 2025,

https://www.f5.com/company/blog/preparing-applications-and-apis-for-generative-ai-with-json-ld
24. JSON-LD Best Practices, accessed June 28, 2025, https://w3c.github.io/json-ld-bp/?specStatus=ED
25. What Should I Know About JSON-LD as a Web Developer? - Wisp CMS, accessed June 28, 2025, https://www.wisp.blog/blog/what-should-i-know-about-json-ld-as-a-web-developer
26. Building JSON-LD APIs: Best Practices, accessed June 28, 2025, https://json-ld.org/spec/latest/json-ld-api-best-practices/
27. Using JSON-LD - linkml documentation, accessed June 28, 2025, https://linkml.io/linkml/howtos/using-jsonld.html
28. Ethereum Sidechains vs Layer 2 Solutions: Find the Differences - Metana, accessed June 28, 2025, https://metana.io/blog/ethereum-sidechains-vs-layer-2-solutions-find-the-differences/
29. Holochain vs IPFS vs Ethereum : r/holochain - Reddit, accessed June 28, 2025, https://www.reddit.com/r/holochain/comments/p46yvh/holochain_vs_ipfs_vs_ethereum/
30. Immutability - IPFS Docs, accessed June 28, 2025, https://docs.ipfs.tech/concepts/immutability/
31. Blockchain IPFS: Ultimate Guide to Decentralized Storage |2024 - Rapid Innovation, accessed June 28, 2025, https://www.rapidinnovation.io/post/blockchain-ipfs-comprehensive-guide-to-decentralized-storage-solutions
32. From IPFS to Ceramic: The future of Web3 data network - Medium, accessed June 28, 2025, https://medium.com/@Web3comVC/from-ipfs-to-ceramic-the-future-of-web3-data-network-f1a16dd9e847
33. Cost-Effective and Reliable Sidechain Approach for Managing Small ..., accessed June 28, 2025, https://www.mdpi.com/2076-3417/15/9/5221
34. Ceramic, accessed June 28, 2025, https://www.diadata.org/web3-ai-map/ceramic/
35. How it works - Ceramic Network, accessed June 28, 2025, https://ceramic.network/how-it-works
36. Overview - Ceramic documentation, accessed June 28, 2025, https://developers.ceramic.network/docs/protocol/ceramic-one
37. The Composable Data Network | Ceramic documentation, accessed June 28, 2025, https://developers.ceramic.network/docs/introduction/intro
38. orbitdb-archive/ipfs-log: Append-only log CRDT on IPFS - GitHub, accessed June 28, 2025, https://github.com/orbitdb-archive/ipfs-log
39. eqlabs/ipfs-log-rs - GitHub, accessed June 28, 2025, https://github.com/eqlabs/ipfs-log-rs
40. A prototype implementation of a healthcare HIPAA audit log using ethereum blockchain and meteor - GitHub, accessed June 28, 2025,

https://github.com/chafey/ethereum-hipaa-audit-log
41. How DLT Regulates Rules and Boosts Market Growth - Number Analytics, accessed June 28, 2025, https://www.numberanalytics.com/blog/dlt-regulation-market-growth
42. Industry News 2024 Beyond the Blockchain Bubble Distributed Ledger Technology for a Resilient Audit Landscape - ISACA, accessed June 28, 2025, https://www.isaca.org/resources/news-and-trends/industry-news/2024/beyond-the-blockchain-bubble-distributed-ledger-technology-for-a-resilient-audit-landscape
43. Verifiable Credentials Data Model v2.0 - W3C, accessed June 28, 2025, https://www.w3.org/TR/vc-data-model-2.0/
44. Verifiable credentials - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Verifiable_credentials
45. Verifiable Credentials Overview - W3C, accessed June 28, 2025, https://www.w3.org/TR/vc-overview/
46. Verifiable Credentials (W3C) - walt.id Docs, accessed June 28, 2025, https://docs.walt.id/community-stack/concepts/digital-credentials/verifiable-credentials-w3c
47. Verifiable Credentials: A Deep Dive for the Agentic AI Era - Shankar's Blog, accessed June 28, 2025, https://shankarkumarasamy.blog/2025/02/28/verifiable-credentials-a-deep-dive-for-the-agentic-ai-era/
48. Self-Sovereign Identity: The Ultimate Guide 2025 - Dock Labs, accessed June 28, 2025, https://www.dock.io/post/self-sovereign-identity
49. Principles of SSI V3 - Sovrin Foundation, accessed June 28, 2025, https://sovrin.org/principles-of-ssi/
50. Explaining W3C Verifiable Credentials and biometrics a key mission for Dock Labs, accessed June 28, 2025, https://www.biometricupdate.com/202505/explaining-w3c-verifiable-credentials-and-biometrics-a-key-mission-for-dock-labs
51. Concept drift - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Concept_drift
52. Drift Detection in Large Language Models: A Practical Guide | by ..., accessed June 28, 2025, https://medium.com/@tsiciliani/drift-detection-in-large-language-models-a-practical-guide-3f54d783792c
53. Tackling data and model drift in AI: Strategies for maintaining accuracy during ML model inference - ResearchGate, accessed June 28, 2025, https://www.researchgate.net/publication/385603249_Tackling_data_and_model_drift_in_AI_Strategies_for_maintaining_accuracy_during_ML_model_inference
54. Behavior Anomaly Detection: Techniques & Best Practices - Exabeam, accessed June 28, 2025, https://www.exabeam.com/explainers/ueba/behavior-anomaly-detection-techniques-and-best-practices/
55. An Entropy-Based Approach for Anomaly Detection in Activities of Daily Living in

the Presence of a Visitor - PMC, accessed June 28, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC7517444/
56. www.researchgate.net, accessed June 28, 2025, https://www.researchgate.net/publication/221611949_An_empirical_evaluation_of_entropy-based_traffic_anomaly_detection#:~:text=The%20entropy%2Dbased%20analysis%20is,of%20unsupervised%20machine%20learning%20algorithms.
57. An empirical evaluation of entropy-based traffic anomaly detection ..., accessed June 28, 2025, https://www.researchgate.net/publication/221611949_An_empirical_evaluation_of_entropy-based_traffic_anomaly_detection
58. Audit Trails in Digital Forensics - Number Analytics, accessed June 28, 2025, https://www.numberanalytics.com/blog/ultimate-guide-audit-trails-digital-forensics
59. What is Audit Trail ? Types & Importance, accessed June 28, 2025, https://www.centraleyes.com/glossary/audit-trail/
60. The Ultimate Guide to Cybersecurity Dashboard UI/UX Design, accessed June 28, 2025, https://www.aufaitux.com/blog/cybersecurity-dashboard-ui-ux-design/
61. Forensic Analysis: Forensic Analysis: Unraveling Mysteries with Audit Trails - FasterCapital, accessed June 28, 2025, https://fastercapital.com/content/Forensic-Analysis--Forensic-Analysis--Unraveling-Mysteries-with-Audit-Trails.html
62. Digital Evidence Dashboard - DFRWS, accessed June 28, 2025, https://dfrws.org/sites/default/files/session-files/2016_EU_pres_the_digital_evidence_dashboard_project.pdf
63. Forensic Audit Guide - Definition, Steps, Reasons - Corporate Finance Institute, accessed June 28, 2025, https://corporatefinanceinstitute.com/resources/accounting/what-is-a-forensic-audit/
64. Zero-knowledge proof - Wikipedia, accessed June 28, 2025, https://en.wikipedia.org/wiki/Zero-knowledge_proof
65. Zero-knowledge proofs explained in 3 examples - Circularise, accessed June 28, 2025, https://www.circularise.com/blogs/zero-knowledge-proofs-explained-in-3-examples
66. What is Zero Trust Zero-Knowledge Proof? - ShareID, accessed June 28, 2025, https://www.shareid.ai/blog/what-is-zero-trust-zero-knowledge-proof
67. Zero-Knowledge Proof Vulnerability Analysis and Security Auditing - Cryptology ePrint Archive, accessed June 28, 2025, https://eprint.iacr.org/2024/514.pdf
68. What is an Incident Response Playbook? - Palo Alto Networks, accessed June 28, 2025, https://www.paloaltonetworks.com/cyberpedia/what-is-an-incident-response-playbook
69. LetsDefend/incident-response-playbooks - GitHub, accessed June 28, 2025, https://github.com/LetsDefend/incident-response-playbooks
70. Incident response playbooks | Microsoft Learn, accessed June 28, 2025,

https://learn.microsoft.com/en-us/security/operations/incident-response-playbooks